Annex XVIII — WR Connect™

Governed Publisher Semantics and Personal Compute Orchestration

Version 1.0 — 10 August 2026 · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure

Annex number provisional — assign per Canon sequence.

---

§XVIII.0 Purpose, Status, and Claims Discipline

§XVIII.0.1 Purpose

This annex discloses how a verified publisher can provide the meaning of its own application — its regions, entities, navigation, workflows, capabilities, and interface anchors — as governed, declarative, cryptographically committed material, and how a local user orchestrator consumes that material to assist, navigate, automate, and dispatch computation without the publisher gaining insight into the user's private context.

A central design goal runs through the entire annex: **enabling low-power local AI to operate accurately.** The publisher side applies strong reasoning once, during preparation, to optimize the semantic environment precisely for consumption by lightweight local models — so that accuracy on the user's device comes from the prepared environment, not from local model size.

§XVIII.0.2 Claims Discipline

This annex is a defensive prior-art disclosure, not a novelty claim. To the author's best knowledge, the individual technologies combined here exist in prior work — including publisher-declared machine semantics, capability-declaration protocols for AI clients, centrally maintained UI semantics in automation tooling, guidance overlays, hybrid on-device/cloud model execution, and computation offloading to nearby or stronger compute. No novelty is claimed for any individual element, and no element-by-element differentiation against specific prior systems is asserted anywhere in this document; this single acknowledgment covers the entire annex.

The disclosed contribution is not any individual element. It is the **combination of these elements under a single cryptographic proof discipline** — WR Handshake™, BEAP™, PoAC™, PoAE™ — with four authorities that never collapse into one another: the publisher is authoritative for declared meaning, the user orchestrator for private meaning, the dispatcher for execution location, and the local trusted UI for everything the user sees.

§XVIII.0.3 Amendment Discipline

Amendments follow the Canon-wide rule: versioned updates, never silent replacement; changes to invariants are explicit and reasoned in the Version History (§XVIII.12).

---

§XVIII.1 Problems Addressed and Value Statement

This section states the concrete problems the architecture solves, before any mechanism is defined. The scenarios are stated briefly in §XVIII.8; the technical realization is specified in §§XVIII.3–XVIII.7.

§XVIII.1.1 Support by local AI instead of hosted support chat

Today, application support is either human (expensive, slow) or a publisher-hosted chatbot (every question, and often the user's account context, flows to the publisher or its AI vendor). With a Publisher Semantic Layer, the publisher describes once — authoritatively — what its application means and how its workflows are structured. The user's **local** AI answers support questions and guides the user through the application using verified publisher semantics, on the user's device. The publisher reduces support cost without operating a support AI; the user receives assistance without disclosing questions, history, or account context to anyone.

§XVIII.1.2 Verified navigation assistance

"Show me where I can download my invoice." "How do I get to returns?" Today an assistant must guess this from pixels or DOM structure, and guesses wrong. With declared semantics, the WR Pointer™ resolves a semantic target from the publisher's own declaration and the WR Overlay™ marks the path — deterministically, from the authoritative source, with a fail-closed rule when the live interface and the declared semantics diverge (§XVIII.6.3).

§XVIII.1.3 Contact channel by BEAP instead of e-mail and web forms

Contact forms and contact e-mail addresses are today's dominant inbound fraud surface: spam, phishing, payload attachments, spoofed senders. A publisher that declares a contact capability in its semantic layer receives inbound contact as **BEAP capsules over the handshake relationship** instead: text-pure by construction (no executable payloads can exist in BEAP text), sender-bound to a handshake identity class (ID0–ID3 per Annex XIV), and screened by Fraud Watchdog™ on ingest. Phishing risk is mitigated structurally, by consistent out-of-band routing — contact travels over the sealed handshake relationship instead of clickable mail and open web forms — combined with Fraud Watchdog™ analysis of every inbound item. The user gains a verified channel to a cryptographically identified publisher; the publisher gains structured, classified, fraud-screened inbound and can retire its most abused legacy channel.

§XVIII.1.4 Conversion through declared registration and low-friction workflows

Registration friction destroys conversion. The user addressed here already holds a WR account but has no account with the publisher. Conventionally, registering means typing personal information into a form and then clicking a link in a confirmation e-mail. Both steps can be performed by the orchestrator on the user's behalf: the publisher declares a registration capability with typed, declared operator-input slots; the orchestrator resolves those slots from the user's private context under identifier-only slot discipline and, after a single consent, submits them and completes the confirmation step — the confirmation e-mail arrives through the governed depackaging path, and its completion is a declared, consented action, never a hidden link click. When the publisher is itself in the WR system, the flow collapses further: the cryptographic account binding of the Public Handshake™ (Annex XIV) and the declared semantic graph make registration fully deterministic

— no form, no confirmation e-mail, one consented action end to end. The conversion benefit accrues to the publisher; the data-minimization benefit accrues to the user — the publisher receives exactly the declared slots, nothing else.

§XVIII.1.5 Maintenance economics

When a publisher changes its interface, today every automation, every client-side agent, and every user re-derives meaning independently — or breaks. Here, the semantic representation is corrected **once, at the source**, published as a new catalog epoch, and propagated to all consumers. Semantic maintenance cost becomes O(1) per interface change instead of O(consumers).

§XVIII.1.6 Accessibility (informative)

A Publisher Semantic Layer is, functionally, a signed, publisher-authoritative accessibility tree plus workflow state. Voice-driven navigation over verified rather than guessed semantics is a direct accessibility capability and is noted here as an intended application domain.

---

§XVIII.2 Definitions

WR Connect™. The collective designation for the architecture disclosed in this annex: the governed publisher connection through which declared application semantics, Context Frames, and the contact and registration capabilities of §XVIII.1 flow between a verified publisher and the local user orchestrator. The publisher-side **WR Connect™ script** (`wr-connect`) is the reference embodiment of the attested publisher channel on ordinary web hosting; it is an embodiment, not the definition — any implementation satisfying the properties of this annex is WR Connect™.

Publisher Semantic Layer (PSL). A declarative, machine-readable representation of the meaning of a publisher's application: semantic regions, entities and relations, navigation structure and transitions, workflow states, capabilities and their preconditions, Semantic Anchors and positional references, and permitted WR Overlay™ declarations. A PSL is publisher material in the full Canon sense (§XVIII.3.1). The PSL is a generalization of the Scope Graph of Annex XIV; it introduces no parallel distribution, versioning, or trust mechanism.

Semantic Anchor. A declared reference to a UI element or region within the publisher's own application, used by the WR Pointer™ and WR Overlay™. *Terminology note:* a Semantic Anchor is distinct from the **WR Anchor** of Annex VIII, which designates an export integration point on a target page. The two terms are intentionally different and must not be conflated.

Local Semantic Environment Model (LSEM). The persistent, local, provenance-classed graph in which the user orchestrator merges validated publisher semantics with locally derived and user-provided meaning (§XVIII.4). The LSEM never leaves the user's trust domain.

Provenance classes. `PUBLISHER_DECLARED`, `PUBLISHER_AI_INFERRED`, `LOCAL_INFERRED`, `USER_PROVIDED`, `PLATFORM_ATTESTED` (§XVIII.4.2).

Context Frame. A short-lived, signed, displayable runtime object describing the currently active semantic context of a session (e.g., "checkout of verified publisher P, step 2 of 4"). Context Frames travel over the handshake channel and are never catalog artifacts (§XVIII.3.5).

WR Overlay™. The branded designation for the orchestrator-rendered assistance and automation surface presented over or alongside an application. Technically, this surface is what the Canon describes as the *Augmented Overlay*: "Augmented Overlay" remains the descriptive technical term for the construct, and **WR Overlay™ is the brand name for it** — chosen because it also conveys the governed automation the surface carries, not augmentation alone. Both terms refer to the same construct; wherever Canon documents use the technical term "Augmented Overlay", the branded designation WR Overlay™ applies equally. All properties, invariants, and governance defined for the Augmented Overlay (in particular Annex XI) apply to the WR Overlay™ unchanged.

WR Link™. A declared, consented automation trigger embedded in or referenced from publisher content: activating a WR Link™ executes a pre-declared AI automation capability on the page, based on WCR recordings, under full capability, consent, and PoAE™ governance. WR Link™ declarations are part of the PSL (§XVIII.3.7).

Personal Compute Dispatcher. The orchestrator component that places tasks among devices within the user's own or explicitly authorized compute environment (§XVIII.7). The governing principle is **execution-location-aware orchestration**: the interaction endpoint does not have to be the execution endpoint.

Interaction Endpoint / Execution Endpoint. The device the user interacts with, and the device on which a given task actually executes. The two may differ; the difference is governed, visible to the user, and invisible to publishers.

---

§XVIII.3 The Publisher Semantic Layer

§XVIII.3.1 Semantics as governed publisher material

A PSL is publisher material in the full sense of this Canon: signed under the publisher's key hierarchy, committed under the Catalog Head with anti-rollback (Annex XIV §XIV.5.5), countersigned at ingest into the Workflow Ready Cloud (WRC)™, publicly auditable, and versioned by catalog epoch. Publication is attested handover per Annex XVII §XVII.3.3 — atomic per epoch, rejection never modification. No separate trust, distribution, or versioning mechanism exists for semantics. A semantic assertion carries the same evidentiary burden as an automation artifact.

§XVIII.3.2 Strictly declarative transport

The logical representation of a PSL is a graph. Its transport form is strictly declarative structured text within BEAP/pBEAP capsules, subject to BEAP text purity. A PSL MUST NOT carry executable content of any kind — no scripts, no WASM, no code in any encoding. The receiving orchestrator (1) receives, (2) validates, (3) admits under PoAC™, (4) reconstructs the graph internally, and (5) merges it into the LSEM under §XVIII.4. Data and declarative semantics cross the boundary; code never does.

§XVIII.3.3 Asymmetric Preparation and free choice of authoring method

Computationally expensive semantic analysis of an application — by strong publisher-side or cloud models — is performed once, on the publisher side, before interaction. Strong intelligence prepares

the environment; lightweight local intelligence operates within the prepared environment. The output of that preparation crosses the trust boundary only in the form defined in §XVIII.3.2.

The purpose of this asymmetry is explicit: **a resource-constrained local model can operate accurately because it no longer has to reconstruct the meaning of an application from raw interfaces.** The hard reasoning has been performed once, publisher-side, with strong models, and its output — declared routes, anchors, capabilities, and the Q&A scenario corpus of §XVIII.3.7 — is optimized precisely for consumption by lightweight local models. Open-ended interpretation, where small models fail, is turned into selection and lookup over validated declarations, where small models are reliable.

How a publisher produces and maintains its semantic layer is the publisher's free choice: manually authored, generated by publisher-side local AI, generated by cloud AI, or any mix of these — including manual authoring of critical declarations with AI-generated coverage of the rest. The governance is method-agnostic: signing, catalog commitment, ingest, countersignature, and audit apply identically regardless of how the artifacts were produced. The only normative consequence of the authoring method is provenance labeling: AI-generated semantics that the publisher has not reviewed MUST be labeled `PUBLISHER_AI_INFERRED`; semantics the publisher has authored, or has explicitly reviewed and adopted as its own declaration, carry `PUBLISHER_DECLARED`. Presenting unreviewed AI inference as declaration violates provenance discipline. *Informative:* the design goal of accuracy-by-preparation is normative in this annex; the magnitude of the resulting capability uplift of small local models is an empirical, measurable, and falsifiable thesis, not an architectural guarantee.

§XVIII.3.4 Central maintenance, update, and invalidation

The publisher-side WR Connect™ script (an embodiment of the attested publisher channel, §XVIII.2) MAY observe the publisher's own application for structural change, regenerate affected semantic artifacts, and publish them as a new catalog epoch. Scalability, update behavior, versioning, cacheability, and invalidation are answered entirely by existing Canon mechanisms and are not redefined here: Catalog Head epochs with anti-rollback (versioning, invalidation), declared load classes entry-class/selection-class with the declared cache vocabulary (cacheability), and delta synchronization over the handshake channel (update propagation).

§XVIII.3.5 Static semantics versus live state

Static meaning — structure, navigation, capabilities, anchors — is catalog material under §XVIII.3.1. Live state — the user's current position in a workflow — is runtime material: short-lived, signed Context Frames over the handshake channel, mirroring the Canon's static/live boundary (Annex XIV). Context Frames confer no authority; they inform. Their availability follows the three-party availability model: without a live publisher orchestrator, static semantics remain fully usable from cache; Context Frames are simply absent.

§XVIII.3.6 Channel honesty

The channel carrying semantic updates and Context Frames is the **handshake-bound direct channel**. Where infrastructure permits, it is peer-to-peer; on constrained publisher hosting it is cryptographically the same sealed channel over a polled dumb carrier. This annex claims channel binding and sealing, not a universal P2P transport.

§XVIII.3.7 Preparation output inventory

The preparation process (§XVIII.3.3) produces a defined set of artifacts. This inventory is explicit and normative in kind — every item is a declarative, provenance-labeled, catalog-committed artifact class; the list is extensible per §XVIII.11, never at runtime:

1. **Navigation Graph** — the backbone: nodes are semantic regions, pages, and workflow steps; edges are declared transitions between them.
2. **Entity and relation declarations** — objects of the application, their relations, and the preconditions of actions upon them.
3. **Capability declarations** — every offered capability, including the registration capability (§XVIII.1.4) and the BEAP contact form capability (§XVIII.1.3) with its typed slots and routing.
4. **Semantic Anchors and position markers** — the declared placement material for WR Overlay™ elements and navigation guidance (§XVIII.5.4).
5. **Semantic navigation aids** — precomputed, declared routes to semantic targets ("path to returns", "path to invoice download"), consumed by the WR Pointer™.
6. **Q&A scenario corpus** — during preparation, the authoring AI systematically plays through the plausible question-and-answer scenarios of the application (account, orders, returns, billing, workflows) and stores the validated scenarios with references into the Navigation Graph, so that the local model answers from prepared, publisher-grounded material instead of improvising.
7. **FAQ set** — the publisher's curated FAQ as declared content, distinct from the AI-generated scenario corpus by provenance label.
8. **WR Link™ declarations** — declared automations, bound to hash-identified WCR recordings, executable on the page via WR Link™ under capability, consent, and PoAE™ governance; loaded by reference at selection per the Canon's selection-scoped loading.
9. **Load, cache, and epoch declarations** — the existing Canon vocabulary (entry-class/selection-class, cache hints, catalog epoch) applied to all of the above.

§XVIII.3.8 Reference implementation pipeline (informative)

An illustrative end-to-end embodiment, without restricting alternatives:

1. **Publisher-side working store.** The WR Connect™ script maintains the semantic layer in a local SQLite database: tables for graph nodes and edges, Semantic Anchors and position markers, declared routes of the publisher's own application (page URLs, form targets — the publisher's public surface, never secrets), Q&A scenario entries, FAQ entries, and capability and WR Link™ declarations with their artifact hashes.
2. **Signed export.** The store is serialized deterministically into canonical declarative JSON, signed under the publisher key hierarchy, committed under the Catalog Head, and ingested with countersignature (§XVIII.3.1).
3. **Delivery.** Artifacts and updates reach the local orchestrator as BEAP/pBEAP capsules over the handshake-bound channel (§XVIII.3.6) — text-pure, sealed, code-free.
4. **Local admission and merge.** The orchestrator validates, admits under PoAC™, and merges into the LSEM under the provenance discipline of §XVIII.4.2.
5. **Local PII enrichment.** Only now, and only locally, is the material enriched with the user's private context — names, addresses, order references, personal synonyms — as

`USER_PROVIDED`/`LOCAL_INFERRED` assertions. PII never travels to the publisher or the WRC™; declared slots are resolved with private values at use time under identifier-only slot discipline.

---

§XVIII.4 The Local Semantic Environment Model

§XVIII.4.1 Purpose and position

The LSEM is the orchestrator's persistent semantic model of the applications and environments the user interacts with: meaning, objects, relations, navigation paths, capabilities, workflow knowledge, Semantic Anchors, overlay declarations, and locally added user information. Its advantage over per-request screen/DOM/LLM parsing is structural: parsing is repeated, probabilistic, and stale the moment it completes; the LSEM is validated once, authoritative at declaration level, incrementally updated by epoch, and queryable in milliseconds without a model in the loop. *Informative embodiment:* a relational store with node/edge tables, recursive traversal, and full-text indexing is sufficient; vector search is optional and not a core requirement.

§XVIII.4.2 Provenance classes and merge discipline

Every assertion in the LSEM carries exactly one provenance class:

- `PUBLISHER_DECLARED` — asserted by the publisher as declaration;
- `PUBLISHER_AI_INFERRED` — produced by publisher-side AI preparation and labeled as such;
- `LOCAL_INFERRED` — derived by local models or heuristics;
- `USER_PROVIDED` — supplied or corrected by the user;
- `PLATFORM_ATTESTED` — ingest countersignatures and platform scanning verdicts.

Local enrichment may extend but MUST NOT silently override publisher-declared assertions. Conflicts are represented as coexisting assertions with distinct provenance, resolved at use time under local policy, with the resolution disclosable to the user on request.

§XVIII.4.3 Lifecycle: epochs, revocation, purge

Provenance classes are the unit of lifecycle management. A catalog epoch advance invalidates and replaces `PUBLISHER_*` assertions selectively; local classes are preserved. Revocation of a WR Handshake™ triggers the declared retention/purge policy for that publisher's derived assertions and for local assertions derived solely from them. The handling of `LOCAL_INFERRED` assertions that build on superseded publisher epochs (retain as unconfirmed, re-validate, or discard) is a declared local policy choice; see Open Items.

§XVIII.4.4 Privacy by architecture

The publisher is authoritative for what its application means; it does not thereby learn what that meaning is worth in the user's private context. Private resolution — personal synonyms, preferences, files, mail, calendar, history, session context, private entity resolution, priorities, final

parameter binding — occurs locally. Governing statement: **public intelligence is prepared globally; private meaning is resolved locally.**

Two consequences are normative:

1. **Query privacy.** Selective loading of semantic contexts is itself a signal. The Canon's default of no selection telemetry extends to PSL loading; load-class declarations and bundling strategies exist precisely so that fine-grained interest is not observable per request.
2. **LSEM protection.** The LSEM is structurally a record of the user's application world and is treated as high-sensitivity local data: encrypted at rest, compartmentalized per publisher, egress-governed by WR Guard™ like any other confidential store.

---

§XVIII.5 WR Overlay™ and Trusted Local Rendering

§XVIII.5.1 Declared, never delivered

Publishers never transmit user interface. Overlay behavior is declared: a declaration names a Semantic Anchor, a meaning, a target reference, and one overlay type drawn from a **closed, locally defined overlay-type catalog**. Rendering is performed exclusively by the orchestrator's trusted UI library. Publishers parameterize within the declared type; they never control presentation. This is the trust boundary between *publisher intent / semantic declaration* and *local trusted rendering*, and it deliberately inverts the model in which a publisher renders guidance into its own page. The visual material of overlays comes exclusively from the sources defined in §XVIII.5.4.

§XVIII.5.2 Invariants

1. **Non-occlusion.** A WR Overlay™ MUST NOT obscure, replace, or visually alter consequential original UI.
2. **Distinguishability.** Overlays are unmistakably marked as orchestrator-rendered, consistent with the Canon's marking obligations.
3. **Fail-closed anchoring.** If a Semantic Anchor does not resolve unambiguously against the live interface, no overlay is rendered at an assumed position; the unmapped-target fallback of Annex XI §XI.5 applies.
4. **Offer-never-action.** WR Overlay™ inherits Annex XI I2 unchanged: it may inform, indicate, and offer — never act. Consequential actions proceed only through capability, validation, consent, and PoAE™.

§XVIII.5.3 Context transparency

While publisher context is active, the user can see that it is active, from which verified publisher it originates, which semantic context (Context Frame) is in use, and why the orchestrator is making its current suggestions. Suggestions derived from publisher semantics are attributable to their provenance class on request.

§XVIII.5.4 Local overlay media library and positional guidance

WR Overlay™ rendering draws its visual material from exactly two sources:

1. **The local overlay media library** — the orchestrator's own trusted library of presentation primitives: arrows, path indicators, markers, highlights, step badges, and comparable directional and annotative elements. These primitives are local assets; publishers can request them by type (§XVIII.5.1), never supply them.
2. **Normalized publisher media**, optionally: media a publisher has uploaded to the Workflow Ready Cloud (WRC)™, normalized at ingest per the Canon's media pipeline, published hash-pinned, and referenced from overlay declarations by hash. The orchestrator renders such media only on hash match. Raw media never travels over the handshake channel.

Locally composed positional guidance. The PSL declares Semantic Anchors and positional references — where semantic targets live within the publisher's interface. The orchestrator resolves these declarations against the locally observed live interface and composes directional guidance from them entirely on the user's device: an arrow toward the invoice-download control, a highlighted path through a multi-step workflow, ordered step markers along a declared navigation route. The publisher declares *where things are and what the target is*; the guidance element itself — its geometry, placement, and rendering — is composed locally from the overlay media library. Every positioned element is subject to fail-closed anchoring (§XVIII.5.2): if a declared position or anchor does not resolve unambiguously, no guidance is rendered at an assumed location.

---

§XVIII.6 Semantic Authority Boundary and Security Model

§XVIII.6.1 Semantic authority is not consequence authority

Signatures on a PSL prove **origin and integrity, never truth**. A publisher is authoritative for the *declared* meaning of its own application; this authority confers no execution authority, no trust uplift, and no influence on consent. All consequential actions remain governed exclusively by the capability, validation, consent, and PoAE™ chain.

§XVIII.6.2 Attack class: semantic spoofing

A verified but malicious or compromised publisher may declare misleading semantics — for example, labeling a subscription trigger as a download link. Mitigations are structural, not heuristic:

1. **Label-independent consent.** The consent surface presented to the operator MUST derive from the validated action itself (intent hash, WYSIWYE per Annex IX), never from publisher-supplied labels or descriptions. The PSL may improve navigation and explanation; it never supplies the description of what the user authorizes.
2. **Platform scanning.** PSL artifacts are scanned by Fraud Watchdog™ in the WRC™ like all publisher material, under the suspension discipline of Annex XVII §XVII.3.3 (visible, auditable, reversible, never modification).
3. **Fail-closed anchoring** (§XVIII.5.2), which prevents misdirected overlays from being rendered on mismatch.

§XVIII.6.3 Staleness and anchor time-of-check/time-of-use

Declared semantics are epoch-versioned; the live interface is not. Divergence between the committed epoch and the rendered application is an expected condition, not an error path: anchor

resolution is fail-closed, resolution failures are surfaced as declared conditions, and a resolution-failure signal is a legitimate trigger for the publisher to republish (§XVIII.3.4) — without per-user telemetry.

§XVIII.6.4 Overlay redressing

A local renderer over foreign UI can itself become a redressing instrument if declarations steer position or content too far. The invariants of §XVIII.5.2 — closed type catalog, non-occlusion, distinguishability — are the defense and are non-negotiable.

§XVIII.6.5 Honest boundary

The PSL eliminates interface ambiguity, not intention ambiguity. The local model remains responsible for interpreting what the user wants, and can be wrong. Nothing in this annex converts probabilistic intention interpretation into a guarantee; the guarantees end where declared semantics end, and consequence gating exists precisely because interpretation can fail.

---

§XVIII.7 The Personal Compute Dispatcher

§XVIII.7.1 Principle

The interaction endpoint does not have to be the execution endpoint. The Personal Compute Dispatcher places tasks among devices within the user's own or explicitly authorized compute environment: interaction device (typically the smartphone orchestrator, "Opty"), personal hosts, home servers, and — only under explicit External Service Admission per Annex X — designated cloud services. The dispatcher is a component of the personal orchestrator. It is not cloud routing: placement stays inside the user-controlled or user-authorized trust domain.

§XVIII.7.2 Placement modes

- **Automatic Dispatch** is permitted only for task classes the user has pre-authorized and only below a declared consequence floor.
- **Suggested Dispatch** applies otherwise: the orchestrator proposes offloading ("this task would run faster/more capably on your host"); the user confirms or continues locally.
- Escalation follows ratchet discipline: policy and user may tighten placement requirements, never loosen them below declared floors.

§XVIII.7.3 Boundaries between the user's own devices

Locality-independent trust (Annex X) applies **between the user's own devices**: data flowing to an execution endpoint is governed by the same classification, WR Guard™, and egress measures as any boundary crossing. Which data classes may reach which host is declared per host, not assumed from ownership.

§XVIII.7.4 Provenance, verification, return

Every dispatched task carries task provenance; every result carries result provenance — executing device identity, model identity, task hash — and is verified by the originating device before re-

insertion into any publisher or workflow context. Dispatch is interruptible; results return to the originating device. The user can inspect where a task ran, why, with which data classes, under which model, and whether placement was automatic or confirmed.

§XVIII.7.5 Placement privacy

Placement is private. Publishers never learn where computation executed; the placement decision is invisible outside the user's trust domain.

---

§XVIII.8 Combined Operation: Brief Scenarios

Each scenario is stated briefly; the technical substance is in §§XVIII.3–XVIII.7.

Local-AI support. "Why was my order split into two shipments?" — answered locally from the Q&A scenario corpus and Navigation Graph plus private context; heavy analysis dispatched to the personal host if needed. The publisher runs no support AI and learns nothing.

Verified navigation. "OK Opty, show me where I download my invoice." — semantic target resolved from declared routes; WR Pointer™ navigates, WR Overlay™ marks the anchor; fail-closed on mismatch.

BEAP contact form. The declared BEAP contact form replaces web form and mail: text-pure, identity-classed BEAP capsules, screened by Fraud Watchdog™ at the publisher.

One-consent registration. Declared slots resolved from local PII under identifier-only discipline; one consent; confirmation handled as a governed action — fully deterministic through the semantic graph when both sides are in the WR system.

WR Link™ automation. A WR Link™ on the page triggers a declared, WCR-based automation — selected, never synthesized — under consent and PoAE™.

Cross-publisher local join (informative). "Compare my open orders across all shops" — a local graph join across provenance-tagged subgraphs plus private entity resolution; the result remains local; any outbound free text falls under the existing WR Guard™ egress discipline (Annex X).

---

§XVIII.9 Essential Combined Disclosure

The following combination is disclosed as of 10 August 2026:

(a) publisher-declared application semantics as cryptographically governed publisher material under a DNS-rooted catalog commitment chain with ingest countersignature and public auditability; (b) asymmetric publisher-side semantic preparation whose output crosses the trust boundary only as validated declarative artifacts, never as executable content; (c) a persistent Local Semantic Environment Model merging publisher-derived and locally derived meaning under explicit provenance classes, with selective epoch invalidation and revocation purge; (d) strict separation of semantic authority from consequence authority, with consent surfaces derived label-independently from validated actions; (e) WR Overlay™ rendering performed exclusively by the local trusted UI from declared intent, under non-occlusion, distinguishability, and fail-closed anchoring; (f)

execution-location-aware orchestration across user-owned compute with consent-graded placement, dual task/result provenance, and placement privacy toward publishers; and (g) the resulting four-authority separation — *the publisher knows the application; the user orchestrator knows the user; the dispatcher knows where computation should run; the trusted local UI controls what the user sees* — governed end to end by one proof discipline.

Individual elements are acknowledged as existing prior work in §XVIII.0.2, which covers this entire annex. The disclosed contribution is this combination and its governance.

---

§XVIII.10 Open Items

1. **Semantic Anchor resolution contract** — the technical form of anchors (structural hash, path, region geometry, semantic selector) and a graded multi-strategy resolution scheme with declared confidence.
2. **Epoch coupling of local enrichment** — normative default for `LOCAL_INFERRED` assertions built on superseded publisher epochs (retain-as-unconfirmed vs. re-validate vs. discard).
3. **Context Frame specification** — schema, lifetime, signature binding, and its relation to session sync across the user's devices.
4. **LSEM replication** — partitioning and synchronization of the LSEM across the user's own devices when the dispatcher separates interaction and execution endpoints.
5. **Semantic context vocabulary** — the open registry schema for context types (product page, checkout, registration, account, order, returns, support, workflow step) rather than a closed normative list.
6. **Dispatcher task-class taxonomy** — the declared task classes and their default consequence floors.

---

§XVIII.11 Extension Points

- Additional provenance classes beyond §XVIII.4.2, added only with a declared merge rule.
- Additional overlay types in the closed local catalog; publisher-side proposals for new types are admitted as Canon amendments, never at runtime.
- Extension of Context Frames to multi-party sessions (interaction with the group-session work).
- Dispatcher extension to attested non-personal compute under External Service Admission profiles.
- Accessibility profiles binding PSL structures to assistive-technology conventions.

---

§XVIII.12 Version History

- **v1.0 — 10 August 2026.** Initial version, published under the designation **WR Connect™** (the publisher-side `wr-connect` script is its reference embodiment). Defines the Publisher Semantic Layer as governed publisher material generalizing the Annex XIV Scope Graph;

Asymmetric Preparation with explicit accuracy-by-preparation design goal (low-power local AI operates accurately on publisher-side strong-reasoning output), method-agnostic authoring (manual, local AI, cloud AI), and review-based provenance labeling; explicit preparation output inventory (Navigation Graph, Q&A scenario corpus, FAQ, Semantic Anchors/position markers, semantic navigation aids, BEAP contact form capability, WR Link™ declarations bound to WCR recordings) with informative SQLite → BEAP → LSEM reference pipeline and local PII enrichment (§§XVIII.3.7–3.8); the Local Semantic Environment Model with provenance classes and lifecycle; introduces **WR Overlay™** as the branded designation for the Augmented Overlay ("Augmented Overlay" remains the descriptive technical term; both refer to the same construct); Semantic Authority Boundary with the semantic-spoofing attack class and label-independent consent; the Personal Compute Dispatcher with placement modes, dual provenance, and placement privacy; local overlay media library with hash-pinned normalized publisher media and locally composed positional guidance (§XVIII.5.4); brief scenarios (local-AI support, verified navigation, BEAP contact form, one-consent registration, WR Link™ automation); consolidated best-knowledge prior-art acknowledgment in the Claims Discipline (§XVIII.0.2, no element-by-element differentiation); essential combined disclosure.

1 WR Connect™

Governed Publisher Semantics and Personal Compute Orchestration

Optirando™ Canon · Annex XVIII · Defensive Publication / Prior-Art Disclosure · 10 August 2026

Publisher-provided semantics instead of screen guessting.

Strong intelligence prepares the environment — so low-power local AI operates accurately.

2

1 Publisher side → 2 Signed export → 3 Workflow Ready Cloud (WRC)™ → 4 BEAP capsule → 5 User device

WR Connect™ script

SQLite working store

- Navigation Graph
- Q&A scenario corpus
- FAQ
- Semantic Anchors + position markers
- Capabilities
- BEAP contact form
- WR Link™ (WCR-bound)

Authored manually, by local AI, or by cloud AI — publisher's choice.

Canonical declarative JSON — signed, never code

WRC CATALOG
EPOCH 000127
WRC COUNTERSIGNED

Fraud Watchdog™ scan

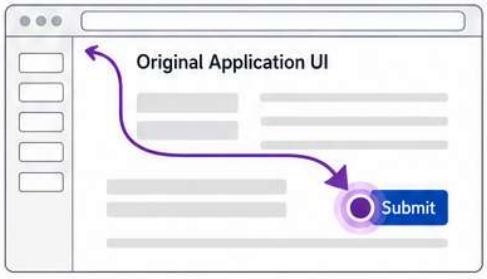
Publicly auditable

Text-pure capsules over the WR Handshake™ channel — data and semantics cross, code never does.

Orchestrator
LSEM
- PUBLISHER_DECLARED
- PUBLISHER_AI_INFERRED
- LOCAL_INFERRED
- USER_PROVIDED

Local PII enrichment — private data never leaves the device.

3 WR Overlay™



- Rendered only by the local trusted UI from the local overlay media library (arrows, paths, markers)
- Publisher declares intent + Semantic Anchors — never delivers UI
- Fail-closed anchoring · Non-occlusion
- WR Pointer™

4

Publisher	User Orchestrator	Dispatcher	Trusted local UI
knows the application	knows the user	knows where computation runs	controls what the user sees

Four authorities that never collapse — governed end to end by WR Handshake™, BEAP™, PoAC™, PoAE™.

5

Opty — interaction endpoint

Automatic Dispatch (pre-authorized task classes)

Suggested Dispatch (user confirms)

Personal host — execution endpoint

The interaction endpoint does not have to be the execution endpoint.
Placement is invisible to publishers.

6

Local-AI support · Verified navigation · BEAP contact form · One-consent registration · WR Link™ automation

Optirando™ · WR Connect™ · WR Overlay™ · WR Pointer™ · WR Link™ · WR Handshake™ · BEAP™ · PoAC™ · PoAE™ · Fraud Watchdog™ · Workflow Ready Cloud (WRC)™ are designations of the Optirando™ Canon.

Opty UI Concept

System-wide Adaptive Pointer Menu + Personal Pinned Actions

The pointer menu adapts to what you're doing right now.
Opty keeps your favorite actions always within reach.

- Tiny & Floating
- Movable
- Dockable
- Always in Reach

1 Wake Opty

1 Sleeping (Pinned icons stay visible)

Opty is sleeping, but your pinned icons remain visible and usable.

2 Activate by Voice (Two ways)

Say to wake Opty: *OK Opty!*

Voice: Say "OK Opty" to to wake.

Say to run an action: *OK Opty, run #Work*

Voice: Say "OK Opty, run #tag" to execute a pinned action.

3 Activate by Click / Tap

Click a pinned icon directly to trigger the action.

4 Opty is Ready

Opty is awake and ready.
Pinned actions are at your fingertips.

⭐ **Pinned icons stay visible, even while Opty sleeps, so you can always wake Opty or run actions by voice or click.**

2 Adaptive Pointer Menu

- Lives in the system-wide pointer menu
- Context-sensitive actions near the cursor
- Color-highlighted Adaptive area
- Based on current target, app, page, handshake, and workflow context
- Some actions can be pinned

VS

Opty (Your Pinboard)

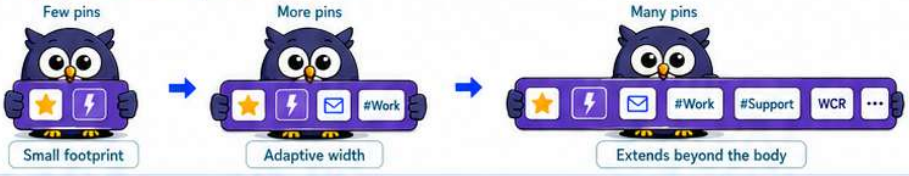
- Tiny floating owl companion
- Freely movable and dockable
- Shows only your pinned actions
- Pins can trigger features, #tags, WCR recordings, and automations
- Shield/pinboard grows horizontally as more pins are added

3 Pointer Menu (Example)

Adaptive area = context-derived suggestions

Same position and accent color — only the actions change based on context.

4 Opty Pinboard Adapts to Your Needs



Docked to edge

5 From Discover to Persistent Shortcut

1 Discover

A useful action appears in the Adaptive area based on current context.

2 Pin

The user pins that action directly from the Adaptive area.

3 Use through Opty

The action becomes a persistent shortcut on your Opty pinboard.

Scope (where it can appear)

Global | App | Publisher/Site | Context/Object

Pinned actions can keep their intended scope.

6 Design Principles

- Adaptive = contextual and temporary
- Opty = personal and persistent
- Stable layout reduces misclicks
- Accent color means context-derived
- Pins should stay compact and glanceable

At a Glance

- **Voice Activation** — Say "OK Opty" to wake Opty.
- **Click Activation** — One click or tap on Opty wakes it.
- **System-wide Pointer Menu** — Context-aware near the cursor.
- **Personal Pinboard** — Your shortcuts, always within reach.
- **Secure & Private** — Your pins stay with you across apps.

Separate contextual actions from personal shortcuts.

The pointer menu adapts to what you're doing now.
Opty keeps your favorite actions always within reach.

Smarter workflows. Faster actions. Better decisions.

Built for your mission. Configured for your success.

Always ready. Whenever you need it.

Optirando